NCCL 2-node inter-node performance: the InfiniBand and PCIe path

Date: 2026-08-07 **Cluster:** AICR B200 (MGHPCC). Nodes b0029, b0030, b0031 — b0029+b0030 are the pair behind `results_b200.md` Table 2 and Table III of `aicr_benchmarks_submitted.pdf`. **Platform:** AMD Turin GPP root complex → Broadcom PEX890xx Gen5 switch → B200 / ConnectX-7 (firmware 28.41.1000), 8 × NDR400 compute rails per node.

Question: AICR's 2-node NCCL collectives run at roughly half the per-rail rate of comparable B200 hardware. Is that a hardware limit or a configuration problem — and if configuration, which setting?

Answer: configuration, not hardware — proven by measurement. The defect is an **ordering/concurrency failure at the GPU PCIe endpoint under bidirectional GDRDMA**. The exact knob is **not yet identified**: the obvious candidate (PCIe relaxed ordering at the memory-region level) was tested and is demonstrably **not** the lever — see §3. The remaining candidates all need root or a healthy-cluster comparison to separate.

---

1. The symptom, in per-rail terms

In a 2-node ring collective NCCL builds one channel per rail, so `busbw` is the aggregate over all 8 NDR rails. Dividing back out gives the physically meaningful number:

	AllGather busbw	÷ 8 rails	SendRecv	Consistent
AICR (b0029+b0030)	218 GB/s	**27.2 GB/s**	**26.6 GB/s**	yes
MIT Engaging (node5500–5502)	383 GB/s	**47.9 GB/s**	**47.8–50.0 GB/s**	yes

Both clusters saturate all eight rails. They differ only in what one rail delivers: **26.6 vs ~48 GB/s**, a factor of 1.8. Engaging is at ~96% of the 50 GB/s NDR nominal rate; AICR is at 53%.

The full collective picture, and the control that makes this a diagnosis rather than a boast:

Collective	AICR	Engaging	Ratio
SendRecv	26.6	47.8–50.0	1.87×
Reduce	201	384	1.91×
Broadcast	202	368	1.82×
AllGather	218	383	1.76×
ReduceScatter	218	382	1.75×
Scatter	293	339	1.16×
AllReduce (Ring)	170	240	1.41×
AllToAll	39.8	47.5–49.9	**1.25×**
Gather	90.5	92.0–95.4	**1.05×**

Everything gated by the fabric roughly doubled. **Gather and AllToAll — the two that are limited by NCCL's algorithms rather than the wire — barely moved.** A faster fabric cannot fix an algorithmic bottleneck, and it did not. That pattern is what says the difference is in the inter-node path, not in NCCL.

Both clusters run effectively the same NCCL: AICR reports `nccl-library=22903` = **NCCL 2.29.3**, *newer* than Engaging's 2.29.2. (The `2.18.3` in the header is the nccl-tests harness version, a common source of confusion.) NCCL version explains none of the gap.

---

2. Isolating the path: six measurements

All run on AICR hardware; none requires root. Scripts: `diag-rootcause.sh`, `diag-gdrdma-ab.sh`, `diag-gdrdma-sides.sh`, `diag-ro-matrix.sh`, `pcie_duplex.cu`. Raw output in `out-diag/` (jobs 300708, 317105, 317112, 317113, 317188).

#	Test	Result	What it establishes
1	**PCIe full duplex**, `cudaMemcpyAsync`, no IB at all	H2D 57.6, D2H 57.3, **concurrent 49.1 each way = 98.3 GB/s total**	the GPU's PCIe link **is** full duplex
2	Host↔Host RDMA, unidirectional	46.3 GB/s	baseline line rate
3	Host↔Host RDMA, **bidirectional**	**47.3 GB/s each way** (94.5 total)	IB fabric, switch uplink and NIC all fine both ways
4	GPU↔GPU RDMA, unidirectional	**47.5 GB/s**	the GDRDMA path is fine one direction at a time
5	GPU↔GPU RDMA, **bidirectional**	**27.2 GB/s each way** (54.5 total)	the defect
6	**one GPU endpoint only**, bidirectional	GPU↔host **30.7**, host↔GPU **27.2** GB/s/dir	a *single* GPU endpoint suffices — per-endpoint, not end-to-end

`ib_write_bw`, rail `mlx5_3`, 8 MiB messages, 2000 iterations. perftest prints bidirectional rows as the **sum** of both directions; the table already halves them.

Test 1 is the one that changes the story. It runs entirely inside a single node using plain `cudaMemcpy` — no InfiniBand, no NIC, no RDMA, no peer-to-peer — and the GPU sustains 49.1 GB/s in *each* direction simultaneously, 98.3 GB/s total. Whatever limits GDRDMA here, it is not a shared transmit/receive budget inside the GPU.

Test 6 is the one that names the mechanism. If the collapse required GPU memory at both ends, a fabric or end-to-end explanation would still be live. It does not: put GPU memory on one side and host memory on the other, and that single endpoint still collapses. The fault is where the NIC's reads and writes meet at one GPU.

Test 5 cross-checks against NCCL exactly: 27.2 GB/s per rail, versus NCCL SendRecv 26.6 and AllGather 218 ÷ 8 = 27.2. The microbenchmark and the collectives are measuring the same wall.

---

3. The mechanism — and an honest correction

The shape of the failure is an ordering/serialization effect: in GPUDirect RDMA the **NIC** is the requester against **GPU BAR** memory — it **reads** GPU memory for outbound traffic and **writes** GPU memory for inbound. One direction at a time,

nothing conflicts and both hit full line rate (tests 2–4). Both at once, the NIC's read completions and posted writes meet at the GPU endpoint and throughput halves (tests 5–6). Strict PCIe ordering — a read completion may not pass a posted write — produces exactly this signature, and relaxed ordering (RO) is the standard cure, documented as essential on AMD platforms by the HPC Advisory Council's EPYC/InfiniBand guidance.

An earlier draft of this document named `/proc/driver/nvidia/params` → `EnablePCIERelaxedOrderingMode: 0` as *the* root cause. **A direct test refuted that as stated.** Two problems:

(a) The MR-level RO knob is demonstrably not the lever. perftest 6.26 registers memory regions with `IBV_ACCESS_RELAXED_ORDERING` **by default** (the flag `--disable_pcie_relaxed` exists to turn it *off*) — so every measurement above already ran with RO requested. Toggling it explicitly (job 317188, same rail, 8 MiB, per-direction figures):

Path, bidirectional	RO on (default)	RO disabled
Host ↔ Host	45.0 GB/s	44.0 GB/s
GPU ↔ GPU	**32.0 GB/s**	**31.5 GB/s**
GPU unidirectional (control)	—	47.4 GB/s

The GPU collapse is identical with RO requested and disabled. Either the RO attribute never reaches the wire for this path — e.g. the NIC's `DevCtl.RlxdOrd` enable bit is cleared by kernel or BIOS, which makes the MR flag a silent no-op — or the defect is not an RO effect at all. (NCCL is in the same position: `libnccl.so` contains `NCCL_IB_PCI_RELAXED_ORDERING` / `IBV_ACCESS_RELAXED_ORDERING` support, so NCCL requesting RO would go through the same possibly-dead path.)

(b) `EnablePCIERelaxedOrderingMode=0` is the NVIDIA driver's default, so healthy clusters very likely show 0 as well. Without reading the same parameter on a healthy B200 system (MIT Engaging), its value here discriminates nothing. That comparison has not been done yet.

What remains standing, and what is open

Standing, on direct measurement: the defect is real, per-GPU-endpoint, bidirectional-only, and platform-level — not silicon, not fabric, not link width, not IOMMU, not the MR-level RO flag.

Open — the specific knob. Remaining candidates, none separable without root or a healthy-node comparison:

1. **NIC `DevCtl.RlxdOrd` cleared at the PCI level** (kernel quirk or BIOS). Would make every software RO request a no-op, consistent with (a). One root `lspci -vvv` settles it.
2. **ConnectX firmware `PCI_WR_ORDERING` = strict** (`mlxconfig` , root). Same effect from the firmware side.
3. **Broadcom PEX890xx switch configuration** for peer-to-peer traffic. The host↔host test never crosses the GPU↔switch leg, so the switch's P2P path is not exonerated by it.
4. **GPU BAR completer behaviour** under mixed inbound writes + read completions — NVIDIA-driver- or vBIOS-level; the `EnablePCIERelaxedOrderingMode` parameter may still matter here, but only the toggle-and-remeasure can say.

Why tests 1 and 5 are not in conflict

They differ in **who is the PCIe requester**, which is what ordering rules key on:

	Requester	Target	Result
Test 1, `cudaMemcpy`	the **GPU's** copy engines	host DRAM	separate engines per direction, no ordering conflict — **98.3 GB/s total**
Test 5, GDRDMA	the **NIC**	**GPU BAR** memory	reads and writes share one ordering domain — **54.5 GB/s total**

Same link, same GPU, same PCIe switch. Only the requester and the applicable ordering rules differ.

4. What was excluded, and how

Candidate	Evidence	Status
B200 silicon / shared TX-RX DMA budget	test 1: 98.3 GB/s total, full duplex	**excluded**
IB fabric / switch / NIC	test 3: 47.3 GB/s each way on the same rail	**excluded**
PCIe link width or generation	all B200s and all 8 compute rails at **Gen5 x16 of x16**	**excluded**
GDRDMA path broken or absent	test 4 at full line rate; `nvidia_peermem` loaded	**excluded**
IOMMU translating P2P	`amd_iommu=off iommu=off` ; `/sys/class/iommu` empty	**excluded**
ACS redirect	`pci=noacs` on cmdline; a root-complex redirect would also cost one-way throughput, which is clean	**excluded**[1]
GPU BAR1 / resizable BAR	BAR1 = 256 GB; Region 2 = 256 G	**excluded**
NCCL version or tuning	AICR runs the *newer* NCCL (2.29.3 vs 2.29.2); tests 1–6 involve no NCCL at all	**excluded**
MR-level relaxed ordering (`IBV_ACCESS_RELAXED_ORDERING`)	perftest requests it by default; toggling it changes nothing (§3a)	**excluded as the lever**
Ordering/concurrency at the GPU endpoint — via NIC `DevCtl.RlxdOrd` , ConnectX `PCI_WR_ORDERING` , PEX890xx switch config, or GPU BAR completer behaviour	signature matches; candidates not separable without root	**the open set**

[1] `ACSCtl` was never actually read. Unprivileged `lspci -vv` silently omits the PCIe Express capability, and `/sys/bus/pci/devices/*/config` returns only 64 of 4096 bytes. Excluded on the kernel cmdline plus the one-way argument, not on a live register read.

Confidence: that this is a fixable platform-configuration defect — high, on direct measurement. That any one specific knob is the culprit — genuinely open; §3 lists four candidates and the root-level checks that separate them.

5. The path to the fix

Two tracks. Track A needs no privileges and may settle it; track B needs root and will.

Track A — no root needed

1. Compare against a healthy B200 node (MIT Engaging node5500–5502). Run `diag-node.sh` and `diag-rootcause.sh` there and diff against `out-diag/diag-b0029.txt` / `out-diag/gdr-root-b0031-317105`, specifically:

```
cat /proc/driver/nvidia/params | grep -i relax    # is Engaging also 0? (driver default)
nvidia-smi -q | grep -i relax                     # "Relaxed Ordering Mode" value
```

If Engaging is healthy with the same values, the NVIDIA driver parameter is fully exonerated and the search narrows to the NIC firmware / switch / kernel-PCI layer.

2. Run the same perftest matrix on Engaging (`diag-ro-matrix.sh`): confirm GPU bidir is ~47 GB/s/dir there, and see whether `--disable_pcie_relaxed` *degrades* it. If disabling RO on Engaging reproduces AICR's collapse, the defect is precisely "RO not reaching the wire on AICR", and the knob is whichever layer differs.

Track B — with root, on an AICR b-node

```
# 1. the registers that decide whether RO ever reaches the wire
lspci -vvv -s <nic-bdf>  | grep -E "DevCtl:|RlxdOrd"   # RlxdOrd+ = device may set RO
lspci -vvv -s <gpu-bdf>  | grep -E "DevCtl:|RlxdOrd"
lspci -vvv | grep -A1 ACSCtl                            # close the ACS gap for good

# 2. NIC firmware ordering policy
mlxconfig -d <nic-bdf> q | grep -i -E "order|relax"     # PCI_WR_ORDERING: per_mkey vs
force_relax

# 3. only if 1-2 are clean: NVIDIA driver parameter toggle
echo 'options nvidia NVreg_EnablePCIeRelaxedOrderingMode=1' \
     > /etc/modprobe.d/nvidia-relaxed-ordering.conf     # confirm spelling via: modinfo
nvidia | grep -i relax
dracut -f && reboot
```

Confirming any fix

Re-run the measurement that found the defect — no interpretation needed:

```
sbatch -w b0029,b0030 diag-gdrdma-ab.sh   # row 4 should move ~27 -> ~47 GB/s/dir
sbatch diag-ro-matrix.sh                   # GPU bidir rows should join the host rows
```

Predicted collective results after a successful fix: SendRecv 26.6 → ~48, AllGather and ReduceScatter 218 → ~380, Reduce 201 → ~380, Broadcast 202 → ~365 GB/s — the Engaging profile.

---

6. Configuration reference

Full dumps: `out-diag/diag-b0029.txt`, `out-diag/gdr-root-b0031-317105`, `out-diag/gdr-root-b0030-317112`.

PCIe path from GPU to root complex:

```
B200 (0000:a3:00.0)
  -> 0000:a2:00.0  Broadcom / LSI PEX890xx PCIe Gen 5 Switch
  -> 0000:a1:00.0  Broadcom / LSI PEX890xx PCIe Gen 5 Switch
  -> 0000:a0:01.1  AMD Turin GPP Bridge
```

Each GPU and its rail NIC sit under the same switch (`PIX` in `nvidia-smi topo -m`), so GDRDMA is switch-local peer-to-peer.

Kernel cmdline: `... pci=pcie_bus_perf ... amd_iommu=off iommu=off pci=noacs ...`

Rails. Every B200 and every compute rail runs width 16 at 32.0 GT/s (Gen5). The four functions at `0000:e3:00.[0-3]` run **x2 at 8.0 GT/s (Gen3)** and report **100 Gb/s** in `ibstat` — these are `mlx5_7/8/9/10`, the management NICs already excluded from the SHARP HCA list. Independent hardware-side confirmation that the exclusion is correct.

Rate	Rails
400 Gb/s, x16 Gen5	`mlx5_0` , `1` , `2` , `3` , `4` , `5` , `6` , `11` , `12`
100 Gb/s, x2 Gen3	`mlx5_7` , `8` , `9` , `10`

Other: `nvidia_peermem` loaded; BAR1 256 GB; ConnectX-7 firmware 28.41.1000; GPU `asyncEngineCount=4` .

7. Method notes

Recorded so a re-run does not repeat these.

The old `test-gdrdma.sh` measures the wrong path. It hardcodes `-d mlx5_0` , which is `NODE` — across a PCIe host bridge — from the GPU at `72:00` , while `mlx5_3` is `PIX` . NCCL uses the PIX rail. `diag-gdrdma-ab.sh` resolves affinity at runtime from `nvidia-smi topo -m` .

Server and client must use the same rail. Letting each end pick its own PIX rail gave `mlx5_3` on one node and `mlx5_12` on the other; those sit on different IB subnets and every test failed with `Failed status 12` (transport retry exceeded). `NIC_FORCE=mlx5_N` pins one rail.

perftest reports bidirectional rows as the sum of both directions. Divide by 2 before comparing with a unidirectional row — otherwise a halved link looks healthy.

`--use_cuda` is per side. That is what makes test 6 possible: mixing GPU and host buffers across the two ends localises the defect to a single endpoint.

Unprivileged `lspci` is silently incomplete. It omits the PCIe Express capability entirely, so grepping for `ACSCtl` or `RlxdOrd` returns nothing — which reads as "clean" but means "not visible". Use the world-readable `/sys/bus/pci/devices/*/{current,max}_link_{width,speed}` instead for link state.

Reproduce:

```
sbatch diag-rootcause.sh                    # 1 GPU: full-duplex test + config dump
sbatch -w b0029,b0030 diag-gdrdma-ab.sh     # 2 nodes: host vs GPU, uni vs bidir
sbatch diag-gdrdma-sides.sh                 # 2 nodes: which endpoint is at fault
```

`b200-devel` (b0029–b0031, 4 h limit) is far less contended than `b200-batch`.

---

8. Consequences

For the paper

The measurements in `aicr_benchmarks_submitted.pdf` are correct; the interpretation is not. Section IV B derives a "≈ 53.5 GB/s HBM budget shared between transmit and receive" and concludes SendRecv's 26.6 GB/s is "a silicon-level wall that no NCCL tuning can overcome." Test 1 refutes this **on AICR's own hardware**: 98.3 GB/s total, 49.1 GB/s each way at once. Section IV D also contradicts IV B directly, quoting Gen5 x16 as "≈ 63 GB/s per direction" and reporting RTX PRO 6000 SendRecv at 37.4 GB/s per direction bidirectionally — above the supposed universal wall.

Needed: drop the shared-budget derivation; recompute the inter-node `%max` column against 50 GB/s per rail / 400 GB/s per node (94–100% becomes ~50–55%); remove the "silicon-level wall" sentence; revise the ~37 ms pipeline-parallel activation budget to ~20 ms; and reword SHARP as working around a configuration limit rather than a physical one.

Unaffected: all intra-node NVLink results, the Gather and AllToAll algorithmic diagnoses, and the two-phase Ring AllReduce explanation — the last of which Engaging actually strengthens (AllReduce/AllGather = 0.63 there vs 0.78 on AICR).

For SHARP

SHARP's measured 2.2× (357 vs 163 GB/s) is **inflated by this defect**. In-switch reduction makes AllReduce single-pass, roughly halving bidirectional PCIe pressure per byte — precisely the pressure that is degraded here. Part of what looks like SHARP's advantage is SHARP routing around a misconfiguration. Expect a smaller gain once relaxed ordering is enabled; for reference, Engaging reaches Ring AllReduce at 240 GB/s with no SHARP at all, versus AICR's 163. The recipe in `notes-sharp.md` is unaffected — it is all runtime configuration — but the headline speedup should be re-measured after the fix.

For users today

Until this is fixed, inter-node bandwidth-bound work on AICR runs at roughly half the hardware's capability. Data-parallel gradient sync, cross-node tensor parallelism and pipeline-parallel activation transfer are all affected in proportion. Keeping collectives inside a node (NVLink, unaffected) is worth more here than it would be on a healthy cluster.

---

9. Review outcome and recommendations

This document was re-checked on 2026-08-07 after its first version named `EnablePCIERelaxedOrderingMode=0` as the root cause. The check falsified that claim (§3). Recording how, because it changes what should be done next.

Why the first claim failed

It rested on two things that felt strong but were not tests:

- **A mechanism that matched the signature.** Strict PCIe ordering does predict "each direction fine alone, halved together." Matching a signature is necessary, not sufficient — several of the four remaining candidates predict the same shape.

- **One config value read in isolation.** `EnablePCIERelaxedOrderingMode: 0` looks damning until you notice it is the NVIDIA driver's **default**. A value equal to the vendor default discriminates nothing without a healthy-system comparison, and that comparison was never made.

The refutation came from asking what measurement would show the claim to be wrong, and running it: perftest already requests RO by default, so toggling it must change the number if RO is the lever. It did not (32.0 vs 31.5 GB/s/dir).

Recommendations

1. **Do not apply the modprobe change as a fix.** Earlier drafts, and `diag-aicr-gdrdma.md`, list `NVreg_EnablePCIeRelaxedOrderingMode=1` as step 1. It may do nothing, and applying it before diagnosing muddies the evidence. Follow §5 order: read registers first, write settings last.
2. **The Engaging comparison is the highest-value next step and needs no privileges** (§5, Track A). Two outcomes are each decisive: if Engaging is healthy with the same NVreg default, the driver parameter is exonerated outright; if `--disable_pcie_relaxed` degrades Engaging to AICR levels, the defect is precisely "RO never reaches the wire on AICR" and the search collapses to whichever layer differs.
3. **`DevCtl.RlxdOrd` on the NIC is the single most informative root-only read.** If that enable bit is cleared, every software RO request — perftest's and NCCL's alike — is a silent no-op, which would explain the toggle result completely and make it the answer.
4. **The conclusions that matter for the paper do not depend on any of this.** Test 1 (98.3 GB/s full duplex, no IB involved) refutes the "shared TX/RX budget" model on AICR's own hardware regardless of which knob is eventually found, and the per-rail arithmetic in §1 stands on measurement alone. §8 can be acted on now.

Two traps worth remembering

- **perftest requests MR-level relaxed ordering by default** (v6.26). A perftest number is not an "RO off" number; the flag `--disable_pcie_relaxed` turns it off.
- **Unprivileged `lspci -vv` silently omits the PCIe Express capability.** Grepping for `ACSCtl` or `RlxdOrd` returns nothing, which reads as "clean" but means "not visible."

Note: `diag-aicr-gdrdma.md` (2026-08-07, earlier the same day) names `EnablePCIERelaxedOrderingMode=0` as the root cause; §3 of this document supersedes that on the strength of the RO-toggle test (job 317188). This file is the current version of record.

Related: `diag-aicr-gdrdma.md` (diagnostic report, partially superseded), `inter-node-nccl.md` (critique of the paper's Section IV B), `notes-sharp.md` (SHARP enablement recipe), `results_b200.md` (original measurements).